AMU | Language, Mind, & Tech
Programming in Python
Ali Asgari

Roadrunner\n

def()

def name(parameter/s,):
"""
Description
"""
#better to have comments
Arguments

Keyword	Purpose
def	Keyword used to initialize a new function.
Name	A unique identifier following the same naming rules as variables.
Parameters	The variable names defined in the () to receive data.
Docstring	The """ Triple quoted """ description of what the function does.
Arguments	The actual values passed into the function when calling it.

def()

```
def automatic_censor(sentence, toxic_word):
    if toxic_word in sentence:
        censored_version = toxic_word[0] + "*" * (len(toxic_word) - 1)
        clean_sentence = sentence.replace(toxic_word, censored_version)
        return clean_sentence
    return sentence

# recalling the function
ucks= ['duck', 'fuck', 'luck']
raw_data = "The duck had the worst luck when it realized the farmer didn't give a fuck about feeding it on time."

final_data = raw_data
for uck in ucks:
    final_data = automatic_censor(final_data, uck)

print(final_data)
```

The d* had the worst l*** when it realized the farmer didn't give a f*** about feeding it on time.**

print()/return

`print()` is like showing off your data
`return` is like having the data

Feature	print()	return
Vibe	"Look at this cool data!"	"Here is the data, do something with it."
Memory	Data is shown and then immediately deleted.	Data is handed back to your main script.
The 'None' Trap	Always results in a NoneType void.	Results in actual, usable data.
Pipeline Status	The End of the Road.	The Bridge to the next step.

imperative /functional programming

Feature	Imperative (The Elephant)	Functional (The Roadrunner)
Logic	Step-by-step instructions (How).	Result-oriented declarations (What).
Tools	for loops, manual counters, empty lists.	map(), filter(), Comprehensions.
State	Mutable (Variables change constantly).	Immutable (Data is transformed, not broken).
Speed	Slow, prone to "off-by-one" human errors.	Optimized, concise, and Pythonic.

imperative /functional programming

```
# THE ELEPHANT (Imperative - Step-by-Step)
# We have to manually build the list and manage the loop state.
raw_tokens = ["  SYNTAX ", "MORPHOLOGY", "  phonetics  "]
clean_tokens = []

for token in raw_tokens:
    stripped = token.strip()
    lowered = stripped.lower()
    clean_tokens.append(lowered)

# THE ROADRUNNER (Functional - Result-Oriented)
# We declare the transformation and apply it to the entire stream at once.
# This is faster, cleaner, and strictly 'Roadrunner' energy.
pro_tokens = list(map(str.lower, map(str.strip, raw_tokens)))

print(f"Elephant Result: {clean_tokens}")
print(f"Roadrunner Result: {pro_tokens}")
```

First-Class Functions

Technical Concept	Scientific Advantage
Assignment	Functions can be stored in variables for cleaner pipelines.
Higher-Order Functions	A function that accepts another function as an input.
Modular Design	Swap processing methods (cleaning vs. tagging) without rewriting code.

```
def whisper(text): return text.lower()
def yell(text): return text.upper()

# Assigning a function to a variable
hush = whisper

# Passing a function as an argument
def apply_style(text, func):
    return func(text)

print(apply_style("DATA", hush)) # Output: data
print(apply_style("data", yell))     # Output: DATA
```

lambda

```
def name(parameter):
expression
return
```

```
lambda parameter : expression
```

Feature	Regular Function (def)	Lambda Function (lambda)
Name	Required and permanent.	Anonymous and disposable.
Structure	Multiple lines and complex logic.	Single expression only.
Use Case	Reusable research pipelines.	Quick, "throwaway" data transformations.
Return	Requires an explicit return.	Automatically returns the expression result.

lambda

```
# THE DEF APPROACH: The 'Slow & Steady' Factory
def clean_id_formal(raw_id):
    """Strips whitespace, lowers case, and adds a research prefix."""
    clean_text = raw_id.strip()
    id_lower = clean_text.lower()
    return f"SUBJ_{id_lower}"

# THE LAMBDA APPROACH: The 'One-Line' Pocket Tool
# Syntax: lambda [input] : [instant_result]
clean_id_fast = lambda text: f"SUBJ_{text.strip().lower()}"

# --- THE TEST ---
participant_input = "  USER_42  "

print(f"Def Machine:    {clean_id_formal(participant_input)}")
print(f"Lambda Hack:    {clean_id_fast(participant_input)}")
```

List Comprehensions

Phase	The "Manual" Loop (The Elephant)	The "Automatic" Comprehension (The Roadrunner)
Space	3-5 lines of setup and execution.	1 line of pure intent.
Logic	"Create list -> Start loop -> Check condition -> Append."	"[Result -> For-Loop -> Condition]"
Speed	Slower (Calling .append() takes time).	Faster (Optimized internally by Python).
Vibe	"Micromanaging the computer's chores."	"Declaring the final state of your data."

[result for item in iterable if condition]

List Comprehensions

```
corpus = ["syntax", "um", "brain", "uh", "neuron", "like", "cognition"]

# --- THE ELEPHANT (Old Version) ---
clean_elephant = []
for word in corpus:
    if len(word) > 4: # Only keeping 'meaningful' long words
        clean_elephant.append(word.upper())

# --- THE ROADRUNNER (List Comprehension) ---
clean_roadrunner = [word.upper() for word in corpus if len(word) > 4]

print(f"Elephant:   {clean_elephant}")
print(f"Roadrunner: {clean_roadrunner}")
# Result: ['SYNTAX', 'BRAIN', 'NEURON', 'COGNITION']
```

Dict Comprehensions

Component	The "Elephant" Loop	The "Roadrunner" Comprehension
Syntax	for x in data: my_dict[x] = y	{key: value for x in data}
Structure	Multi-line setup; manual key assignment.	Single-line; curly-brace {} declaration.

{key: value for item in iterable if condition}

Dict Comprehensions

```
# THE DATA: A list of words and their associated brain-activation scores
brain_data = [("syntax", 85), ("neuron", 92), ("um", 10), ("semantics", 78), ("uh", 5)]

# --- THE ELEPHANT (Manual Laboratory Work) ---
high_activation_old = {}
for word, score in brain_data:
    if score > 50:
        high_activation_old[word] = score

# --- THE ROADRUNNER (Dictionary Comprehension) ---
high_activation_fast = {word: score for word, score in brain_data if score > 50}

print(f"Elephant Map:   {high_activation_old}")
print(f"Roadrunner Map: {high_activation_fast}")
# Result: {'syntax': 85, 'neuron': 92, 'semantics': 78}
```

Dict Comprehensions

Side Quest!

```
# THE DATA: A list of words and their associated brain-activation scores
brain_data = [("syntax", 85), ("neuron", 92), ("um", 10), ("semantics", 78), ("uh", 5)]

# --- THE ELEPHANT (Manual Laboratory Work) ---
high_activation_old = {}
for word, score in brain_data:
    if score > 50:
        high_activation_old[word] = score

# --- THE ROADRUNNER (Dictionary Comprehension) ---
high_activation_fast = {word: score for word, score in brain_data if score > 50}

print(f"Elephant Map:   {high_activation_old}")
print(f"Roadrunner Map: {high_activation_fast}")
# Result: {'syntax': 85, 'neuron': 92, 'semantics': 78}
```

Your reward : 2pt

Ternary Operator

Feature	The Elephant (if/else block)	The Roadrunner (Ternary)
Logic	4 lines of indented code.	1 line of fluid thought.
Identity	A structural control block.	A functional expression.

Formation	if cond:
	  val = X
	else:
	  val = Y
		X if cond else Y

```
ms = 250

# --- THE ELEPHANT (Standard Block) ---
if ms < 300:
   category = "Fast"
else:
   category = "Slow"

# --- THE ROADRUNNER (Ternary Operator) ---
speed_label = "Fast" if ms < 300 else "Slow"

# --- THE VERDICT ---
print(f"Trial Result: {speed_label}")
```

Value_If_True if Condition else Value_If_False

Unpacking

Technique	The Elephant (Indexing)	The Roadrunner (Unpacking)
Method	x = data[0], y = data[1]	x, y = data
Effort	High (Manual counting of positions).	Zero (Instant, logical assignment).
"The Star" (*)	Requires complex slicing logic.	Captures the "rest" of the data instantly.
"The Void" (_)	Data is assigned but clutters memory.	Signal to Python: "Discard this noise."

The Star (*): You use this when you don't know (or don't care) how many items are in the middle, but you might want to use that list later.

The Underscore (_): You use this to tell other programmers, "I am forced by Python's syntax to give this a name, but I am never going to use it."

Unpacking

```
# THE DATA
packet = ["SUBJ_07", 82, 91, 77, 85, "ACTIVE"]

# --- SCENARIO 1: The Basic Unpack ---
id, s1, s2, s3, s4, status = packet

# --- SCENARIO 2: The Star (*) Catch-All ---
p_id, *all_scores, p_status = packet

# --- SCENARIO 3: The Underscore (_) Disposal ---
subject, *_, final_score, _ = packet
```

```
print(f"Roadrunner ID: {p_id}")
print(f"Cleaned Scores: {all_scores}")
print(f"Final Marker: {final_score}")

# Roadrunner ID: SUBJ_07
# Cleaned Scores: [82, 91, 77, 85]
# Final Marker: 85
```

zip()

Feature	The Elephant (Nested Loops)	The Roadrunner (zip())
Method	for i in range(len(list1)): val = list2[i]	for x, y in zip(list1, list2):

```
regions = ["Broca's Area", "Wernicke's Area", "Visual Cortex"]
activations = [88.5, 92.1, 74.3, 12.0] # Notice: This list is longer!

# --- THE ELEPHANT (Manual Indexing) ---
for i in range(len(regions)):
    print(f"Region: {regions[i]} | Score: {activations[i]}")

# --- THE ROADRUNNER (The Zip Stitch) ---
for region, score in zip(regions, activations):
    print(f"Region: {region} | Score: {score}")

# --- Dict Comprehension + Zip ---
brain_map = {r: s for r, s in zip(regions, activations)}
```

```
import string

def elephant_analyzer(text):
    """The Elephant: One... step... at... a... time."""
    # 1. SPLITTING: Break the sentence into a list
    raw_words = text.split()

    # 2. CLEANING: A slow, manual scrub for each word
    cleaned_list = []
    for word in raw_words:
        # Job 1: Remove punctuation
        no_punctuation = word.strip(string.punctuation)
        # Job 2: Convert to lower case
        lower_case_word = no_punctuation.lower()
        # Job 3: Only keep if it's not empty
        if len(lower_case_word) > 0:
            cleaned_list.append(lower_case_word)

    # 3. COUNTING: Building the frequency map manually
    length_tally = {}
    for w in cleaned_list:
        word_size = len(w)
        # Manually checking for key existence (No shortcuts!)
        if word_size in length_tally:
            current_value = length_tally[word_size]
            new_value = current_value + 1
            length_tally[word_size] = new_value
        else:
            length_tally[word_size] = 1

    # 4. PERCENTAGE: Calculating the math step-by-step
    total_word_count = len(cleaned_list)
    final_output = {}

    # We even sort the keys manually by creating a new list
    all_keys = list(length_tally.keys())
    all_keys.sort(reverse=True)

    for k in all_keys:
        count_for_this_length = length_tally[k]
        ratio = count_for_this_length / total_word_count
        percentage = ratio * 100
        final_output[k] = percentage

    return final_output
```

This tool takes raw linguistic data, cleans it, calculates the word length frequency, and provides a percentage breakdown, sorted from the longest words to the shortest percentage.

```
import string

def roadrunner_analyzer(text):
    """The Roadrunner: Total data domination in two pulses."""
    # Pulse 1: Instant cleaning and normalization
    data = [w.strip(string.punctuation).lower() for w in text.split() if w]

    # Pulse 2: The ultimate dictionary build
return {
        f"{l}-Letter ({'COMPLEX' if l > 8 else 'BASIC'})": (data.count(next(w for w in data if len(w) == l)) / len(data)) * 100
        for l in sorted({len(w) for w in data}, reverse=True)
    }
```

46-16=30

QUIZ!

You have the questions; here is the data:
(not needed necessarily but helping you)

```
comment = """OMG! Rockstar finally said GTA 6 is
coming out?
I am literally shaking right now.
The graphics look insane, but I hope my PC can run it.
hype hype hype!"""
```